Legal Case Research System — Full Project Redesign

1. Project Overview & Objectives

1. Project Overview & Objectives

A web-based Legal Case Research & Document Verification System that helps Court Staff, Advocates, Legal Scholars, and Public users search verified case precedents, analyze draft petitions using AI, and ensure document integrity through cryptographic verification.

Problem Statement: Legal professionals waste hours manually searching through case records, cross-referencing statutes, and verifying document authenticity. This system automates that pipeline end-to-end.

Scope (Final Year Project):

- User authentication with Role-Based Access Control (4 roles)
- Document ingestion pipeline (PDF/DOCX → text extraction → PII redaction → indexing)
- AI-assisted legal query understanding and precedent matching
- SHA-256 document integrity verification with immutable audit trail
- Draft petition assistant with citation insertion
- Admin dashboard for case uploads, approvals, and analytics

Out of Scope (v1): Real-time court API integration, multilingual OCR, mobile app.

2. User Roles & Access Control (RBAC)

2. User Roles & Access Control (RBAC)

Four roles with strictly scoped permissions. JWT token carries role claim; backend middleware validates on every API call.

Feature	Court Staff	Advocate	Legal Scholar / Student	Public
Login & Register	✅ Admin-created	✅ Self-register	✅ Self-register	✅ Self-register
Upload Case to Repository	✅	❌	❌	❌
Admin Approval Queue	✅ Senior only	❌	❌	❌
Full-Text Case Search	✅	✅	✅	Basic only
View Full Case Document	✅	✅	✅ Read-only	❌ (Summary only)
Download Verified Copy	✅	✅	❌	❌
AI Draft Assistant	❌	✅	✅	❌
Insert Citations into Draft	❌	✅	❌	❌
View Audit Logs	✅ All users	✅ Own logs	❌	❌
Admin Analytics Dashboard	✅	❌	❌	❌
Bookmark / Save Cases	✅	✅	✅	❌

Role Assignment Flow: Public → self-registers → email verified → role = Public. Advocate / Scholar → registers with Bar Council ID or institution ID → pending verification → Staff approves → role elevated.

3. System Architecture & Tech Stack

3. System Architecture & Tech Stack

Three-tier architecture: React frontend → FastAPI backend → PostgreSQL + vector DB. Chosen for ease of development, free-tier deployment, and strong Python ecosystem for NLP/AI tasks.

Layer	Technology	Why
Frontend	React.js + TailwindCSS	Component-based, fast UI, good for role-based dashboard switching
Backend API	FastAPI (Python)	Async, auto-generates Swagger docs, great NLP library support
Authentication	JWT + bcrypt	Stateless tokens, role claim in payload, easy to implement
Primary Database	PostgreSQL	Relational, handles metadata + user/audit tables cleanly
Document Storage	MinIO (S3-compatible)	Self-hosted object store for PDF/DOCX files
Vector Search	ChromaDB (local)	Stores sentence embeddings for semantic case matching
NLP / AI	spaCy + Sentence-Transformers	Entity extraction (sections, parties) + semantic embeddings
LLM Layer	Anthropic Claude API (claude-sonnet-4-6)	Intent classification, argument gap analysis, draft suggestions
PII Redaction	Presidio (Microsoft, open-source)	Detects Aadhaar, phone, names — redacts before storage
PDF Extraction	PyMuPDF (fitz)	Fast, handles scanned + digital PDFs
Hashing	Python hashlib (SHA-256)	Document integrity verification
Audit Log	Append-only PostgreSQL table	Immutable access log; no UPDATE/DELETE allowed on this table

Layer	Technology	Why
Deployment	Render.com / Railway (free tier)	Easy CI/CD from GitHub for college project demo

Architecture Diagram (simplified):

```
User Browser (React)
       ↓ HTTPS
FastAPI Backend
  ├── Auth Module (JWT)
  ├── Ingestion Pipeline (Extract → PII Redact → Hash → Embed → Store)
  ├── Search Engine (PostgreSQL + ChromaDB)
  ├── AI Layer (spaCy + Claude API)
  └── Audit Logger
       ↓
PostgreSQL | MinIO | ChromaDB
```

4. Step-by-Step Flow — Inch by Inch

4. Step-by-Step Flow — Inch by Inch

Step 1: Registration & Login

1. User visits `/register` → fills Name, Email, Password, Role selection
2. If Advocate/Scholar → additionally uploads credential ID (Bar Council No. / Institution ID)
3. Backend: bcrypt hashes password → stores in `users` table with `status = pending_verification`
4. Email OTP sent → user verifies → `status = active` (Public role activated immediately)
5. Advocate/Scholar: Court Staff logs into admin panel → reviews credential → approves → role elevated
6. Login: POST `/auth/login` → JWT issued (payload: `user_id`, `role`, `exp`) → stored in `localStorage`
7. Every subsequent API call: `Authorization: Bearer <token>` header → FastAPI middleware decodes → role-gated

Step 2: Role-Based Dashboard Load

1. Frontend reads JWT role claim on login success
2. React Router renders the correct dashboard component:
   - Court Staff → Upload Queue + Audit Logs + Analytics
   - Advocate → Search + Draft Assistant + My Cases
   - Scholar → Search + Read-only View + Bookmarks
   - Public → Search bar only + Summary view
3. Sidebar menu items rendered conditionally based on role

Step 3: Court Staff — Case Upload (New Step)

1. Staff clicks "Upload New Case" → form: Case No., Court Name, Year, Judge, Parties, Act/Section tags
2. Uploads PDF/DOCX file → frontend sends multipart form to POST `/cases/upload`
3. Backend saves to MinIO with path `cases/<case_id>/original.<ext>`
4. Ingestion pipeline triggered (see Step 4)
5. Case enters `status = pending_approval` queue
6. Senior Staff sees pending queue → reviews extracted metadata → Approve / Reject with reason
7. On approval: `status = published`, case becomes searchable

Step 4: Ingestion & Pre-Processing Pipeline

1. **Text Extraction:** PyMuPDF extracts raw text page-by-page from PDF; python-docx for DOCX
2. **Duplicate Check:** SHA-256 hash of raw text computed → checked against `document_hashes` table → if match found, reject with "duplicate" error
3. **PII Redaction:** Presidio scans for Aadhaar patterns, phone numbers, personal names in non-party context → replaces with `[REDACTED]` → redacted version stored separately
4. **Legal Entity Extraction (spaCy):** Custom NER model extracts:
   - IPC/CrPC/IT Act section numbers (e.g., "Section 420 IPC")
   - Legal terms (bail, injunction, writ, negligence)
   - Party names, court name, date of judgment
5. **Sentence Embedding:** Full case text → Sentence-Transformers (`all-MiniLM-L6-v2`) → 384-dim vector → stored in ChromaDB with `case_id` metadata

6. **Metadata stored in PostgreSQL:** Case ID, title, court, year, judge, sections[], keywords[], file path, hash, status, uploaded_by, approved_by, timestamps

Step 5: User — Document / Query Input

1. User navigates to Search page
2. **Option A — Natural Language Query:** Types "bail petition for NDPS offence with prior conviction" into search bar
3. **Option B — Document Upload:** Uploads their draft petition PDF/DOCX
   - Same extraction pipeline runs (Steps 4.1, 4.4)
   - Extracted sections/terms become the search query
4. Query sent to POST `/search` with payload: `{query_text, filters: {court, year_range, act, outcome}}`

Step 6: AI Query Understanding (LLM Layer)

1. Raw query text sent to Claude API (claude-sonnet-4-6) with system prompt:

> "You are a legal query classifier. Extract: (1) relevant IPC/CrPC/IT Act sections, (2) legal issue type, (3) jurisdiction. Return JSON."

2. Claude returns structured JSON: `{sections: ["Sec 37 NDPS", "Art 21"], issue: "bail", jurisdiction: "Sessions Court"}`
3. This enriches the search query with explicit section tags even if user didn't mention them
4. Also runs: "Does this draft have missing arguments?" → returns gap list for Advocate view

Step 7: Search & Matching Engine

1. **Vector Search:** User query embedded → ChromaDB cosine similarity search → top 20 semantic matches
2. **Keyword Search:** PostgreSQL full-text search on `sections[]` and `keywords[]` columns → exact section matches ranked highest
3. **Score Fusion:** Combine semantic score (0–1) + keyword match count → weighted final relevance score

4. **Filters applied:** court type, year range, act, outcome (convicted/acquitted) from user filter panel
5. Results sorted by final score descending → top 10 returned

Step 8: Results Display

1. Search results list rendered with cards showing:
   - Case Title, Court, Year, Citation
   - Relevance score badge (High / Medium / Low)
   - Matched sections highlighted in yellow
   - Brief AI-generated summary (2–3 lines)
   - Outcome tag (Allowed / Dismissed / Partly Allowed)
   - Cited-by count (how many other cases reference this)
2. Filter panel on left: Court Type, Year Range, Act, Outcome, Jurisdiction
3. User can Bookmark case, Compare 2 cases side-by-side, or click to open full view

Step 9: Document View & Integrity Verification

1. User clicks a case → GET `/cases/<case_id>/view`
2. Backend: Re-computes SHA-256 of stored document → compares with hash in DB
3. **If match:** Document served with ✅ Green "Verified & Untampered" badge
4. **If mismatch:** ❌ Red "Integrity Violation" alert → admin notified → document quarantined
5. Audit log entry created: `{user_id, case_id, action: "VIEW", timestamp, ip_address}`
6. Matched sections highlighted in document viewer (PDF.js rendered inline)
7. Advocate: Sees "Insert Citation" button next to each paragraph → click inserts `[Case Name, Year, Court]` into their active draft

Step 10: Draft Assistant (Advocate / Scholar)

1. Advocate opens Draft tab → pastes or uploads their petition draft
2. Claude API analyzes draft:
   - Identifies arguments made
   - Flags missing legal grounds based on matched precedents
   - Suggests relevant sections not cited

3. Suggestions shown as inline cards: "Consider citing Sec 161 CrPC re: confession admissibility — 3 similar cases found"
4. One-click: Insert suggested citation into draft
5. Export draft as DOCX with embedded citations and a Reference Table appended

Step 11: Download & Watermarking

1. User clicks Download Verified Copy
2. Backend generates a new PDF copy with:
   - Invisible text watermark: `Downloaded by: <user_id> | <timestamp> | <case_id>`
   - First page header: "Verified copy — SHA-256: <hash> | Downloaded: <date>"
3. Audit log: `{action: "DOWNLOAD", user_id, case_id, timestamp}`
4. File streamed to user as download

Step 12: Admin Analytics Dashboard (Court Staff)

1. Dashboard shows:
   - Total cases in repository, pending approvals, rejected cases
   - Most searched sections this week (bar chart)
   - Most accessed cases (table)
   - User activity by role (line chart)
   - Unusual access alerts (same doc accessed 20+ times by one user in 1 hour)
2. Audit log export: CSV download of all access logs for a date range

5. Database Design & Repository Structure

5. Database Design & Repository Structure

PostgreSQL handles all relational data. MinIO stores raw files. ChromaDB stores vectors. Each system has a single responsibility.

PostgreSQL Tables:

Table	Key Columns	Purpose
`users`	id, name, email, password_hash, role, status, credential_id, created_at	All user accounts
`cases`	id, case_no, title, court, year, judge, parties, act_tags[], section_tags[], keyword_tags[], file_path, file_hash, status, uploaded_by, approved_by, approved_at	Case metadata
`document_versions`	id, case_id, version_no, file_path, file_hash, changed_by, changed_at	Version history per case
`audit_logs`	id, user_id, case_id, action, ip_address, timestamp	Immutable access log (no DELETE)
`bookmarks`	id, user_id, case_id, created_at	User saved cases
`draft_sessions`	id, user_id, draft_text, citations_inserted[], last_updated	Advocate draft workspace
`approval_queue`	id, case_id, submitted_by, reviewed_by, status, review_note, reviewed_at	Staff approval workflow

MinIO Bucket Structure:

```
legal-repository/
├── cases/
│   └── <case_id>/
│       ├── original.pdf          ← raw upload
│       ├── redacted.pdf          ← PII-redacted version
│       └── verified_copy.pdf     ← watermarked download
└── drafts/
    └── <user_id>/draft_<session_id>.docx
```

ChromaDB Collection: `case_embeddings`

- Document: full case text (chunked by paragraph)
- Metadata: `{case_id, court, year, sections, outcome}`
- Embedding: 384-dim vector (Sentence-Transformers)

Audit Log Protection: PostgreSQL Role `audit_writer` has INSERT-only permission on `audit_logs`. Application user never has UPDATE/DELETE on this table. Trigger prevents row modification after insert.

6. Security Layer — Design Decisions

6. Security Layer — Design Decisions

Security is the core differentiator of this system. Five distinct mechanisms work together.

6.1 SHA-256 Document Integrity

- On upload: `hashlib.sha256(file_bytes).hexdigest()` → stored in `cases.file_hash`
- On every view/download: hash recomputed from MinIO file → compared with DB value
- Match → ✅ Green badge served with document. Mismatch → ❌ alert to admin, document quarantined
- Version-aware: each `document_versions` row has its own hash

6.2 PII Redaction (Presidio)

- Runs before any storage or indexing
- Detects: Aadhaar (12-digit patterns), phone numbers, personal email addresses, names outside party context
- Redacted copy stored as separate file — original preserved for staff only
- Public and Scholar users always served the redacted version

6.3 Immutable Audit Trail

- Every action logged: LOGIN, SEARCH, VIEW, DOWNLOAD, UPLOAD, APPROVE, REJECT
- PostgreSQL trigger blocks UPDATE/DELETE on `audit_logs`
- Log fields: user_id, case_id, action, ip_address, user_agent, timestamp
- Admin can export logs as CSV but cannot modify them in-app
- Future upgrade: hash-chain the logs (each row stores SHA-256 of previous row's hash) for tamper-evidence

6.4 Download Watermarking

- PyMuPDF adds invisible metadata watermark to downloaded PDF

- Visible header on page 1: "Verified Copy | User: <masked_id> | Date: <date> | Hash: <first 16 chars>"
- If a leaked document is found, watermark traces back to the downloader

6.5 Role & API Security

- JWT expiry: 8 hours (access token), 7 days (refresh token)
- FastAPI dependency injection: every protected route has `Depends(require_role(["advocate", "staff"]))`
- Rate limiting: 100 requests/minute per IP (slowapi middleware)
- HTTPS enforced; CORS restricted to frontend domain only
- Passwords: bcrypt with cost factor 12

7. AI / NLP Layer — How It Works

7. AI / NLP Layer — How It Works

Two-layer AI: local NLP (spaCy) for fast entity extraction, remote LLM (Claude API) for reasoning and draft analysis.

7.1 Local NLP Pipeline (spaCy)

- Custom NER model trained on Indian legal text (or rule-based patterns for MVP)
- Extracts: section numbers (`Sec 420 IPC` , `Art 21`), legal terms (`bail` , `injunction` , `writ`), party names, court names, dates
- Runs on every document at ingestion time and on every user query
- Fast: runs in <500ms, no API cost

7.2 Semantic Embeddings (Sentence-Transformers)

- Model: `all-MiniLM-L6-v2` (lightweight, 80MB, runs on CPU)
- Every case document chunked into paragraphs → each chunk embedded → stored in ChromaDB
- User query also embedded → cosine similarity search → top 20 semantic matches
- This catches meaning-based matches even when exact keywords don't overlap

7.3 LLM Layer (Claude API — claude-sonnet-4-6)

Used for three tasks only (to control API cost for college project):

Task	When Triggered	Input	Output
Query Enrichment	Every search	User's raw query	JSON: sections[], issue type, jurisdiction
Draft Gap Analysis	Advocate uploads draft	Petition text + top 5 matched cases	List of missing arguments / sections
AI Summary	Case opened	Case full text (truncated to 2000 tokens)	3-sentence summary

7.4 Search Score Fusion Formula:

```
final_score = (0.6 × semantic_score) + (0.4 × keyword_match_score)
keyword_match_score = matched_sections_count / total_query_sections
```

Semantics weighted higher for natural language queries; keyword weight can be tuned via admin config.

8. Frontend Module Breakdown

8. Frontend Module Breakdown

React SPA with role-gated routing. 7 main modules.

Module	Route	Roles	Key Components
Auth	`/login`, `/register`	All	LoginForm, RegisterForm, OTP Verification
Search	`/search`	All (filtered by role)	SearchBar, FilterPanel, ResultCards, CompareView
Case Viewer	`/cases/:id`	Staff, Advocate, Scholar	PDFViewer (PDF.js), IntegrityBadge, HighlightedSections, InsertCitation button
Draft Assistant	`/draft`	Advocate, Scholar	DraftEditor, AIGapAnalysis panel, CitationSuggestions, ExportButton

Module	Route	Roles	Key Components
Upload	`/upload`	Court Staff	UploadForm, MetadataForm, IngestionProgress
Admin Panel	`/admin`	Court Staff	ApprovalQueue, AuditLogTable, AnalyticsCharts, UserManagement
Bookmarks	`/bookmarks`	Staff, Advocate, Scholar	SavedCasesList, FolderOrganizer

Key UI Decisions:

- Search results: Card layout, not table — easier to scan on smaller screens
- Integrity badge: Always visible in case viewer header (Green ✅ / Red ❌)
- Draft editor: Split-pane — left: their petition, right: AI suggestions + search results
- Mobile responsive: TailwindCSS breakpoints; sidebar collapses to hamburger on mobile

9. Backend API Design

9. Backend API Design

RESTful API built with FastAPI. All endpoints return JSON. Auth endpoints are public; all others require Bearer token.

Method	Endpoint	Role	Description
POST	`/auth/register`	Public	Register new user
POST	`/auth/login`	Public	Login, returns JWT
POST	`/auth/verify-otp`	Public	Email OTP verification
GET	`/users/me`	All	Get own profile
POST	`/cases/upload`	Staff	Upload new case document
GET	`/cases/{id}`	All (role-filtered)	Get case metadata
GET	`/cases/{id}/view`	Staff, Advocate, Scholar	View case + integrity check

Method	Endpoint	Role	Description
GET	`/cases/{id}/download`	Staff, Advocate	Download watermarked copy
POST	`/search`	All	Search cases (semantic + keyword)
GET	`/admin/queue`	Staff	Get pending approval queue
POST	`/admin/approve/{case_id}`	Staff (Senior)	Approve/reject case
GET	`/admin/audit-logs`	Staff	Get audit logs with filters
GET	`/admin/analytics`	Staff	Get analytics data
POST	`/draft/analyze`	Advocate, Scholar	AI draft gap analysis
POST	`/draft/export`	Advocate	Export draft as DOCX
POST	`/bookmarks`	Staff, Advocate, Scholar	Save a case
GET	`/bookmarks`	Staff, Advocate, Scholar	Get saved cases

Standard Response Shape:

```
{
  "status": "success",
  "data": { ... },
  "meta": { "request_id": "uuid", "timestamp": "ISO8601" }
}
```

Error Shape:

```
{
  "status": "error",
  "code": "INTEGRITY_VIOLATION",
  "message": "Document hash mismatch detected"
}
```

10. Testing & Evaluation Plan

10. Testing & Evaluation Plan

Three test levels for a college project demo: unit, integration, and a manual evaluation rubric for the AI components.

10.1 Unit Tests (pytest)

- `test_text_extraction.py` — PDF and DOCX extraction accuracy
- `test_pii_redaction.py` — Aadhaar, phone patterns correctly redacted
- `test_hash_integrity.py` — SHA-256 computed correctly; tampered file detected
- `test_jwt_auth.py` — Token generation, expiry, role claim validation
- `test_search_scoring.py` — Score fusion formula returns correct ranking order

10.2 Integration Tests

- Upload → Ingestion → Search → View round-trip test
- Role access: Advocate cannot access `/admin/queue` (expect 403)
- Audit log: Every VIEW action creates a log entry
- Integrity violation: Manually modify a stored file → system returns ❌ badge

10.3 AI Evaluation (Manual Rubric)

Test Case	Input	Expected Output	Pass Criteria
Query Enrichment	"bail for murder accused"	sections: [Sec 439 CrPC, Art 21]	At least 1 relevant section extracted
Draft Gap Analysis	Weak petition missing bail grounds	Suggests Sec 439 + precedent	⩾ 2 relevant suggestions
Semantic Search	"accused denied legal counsel"	Returns Art 22 / Sec 161 CrPC cases	Top 5 results semantically relevant

10.4 Demo Dataset

- Seed 50 real public-domain Indian case judgments (Supreme Court, High Courts) into repository
- Source: indiankanoon.org (public access, educational use)

- Cover: bail, contract, negligence, IT Act, NI Act cases — ensures search demo has varied results

10.5 Performance Targets (for demo)

- Search response: < 2 seconds for semantic + keyword combined
- Document integrity check: < 300ms
- Ingestion pipeline (1 PDF, 20 pages): < 10 seconds end-to-end